SpecMoE: Operator-Level Co-Design of Speculative Decoding and Mixture-of-Experts Inference

Changwu Li
University (Anonymous for Review)

Abstract

Speculative Decoding (SD) is the standard technique for accelerating autoregressive LLM inference, yet applying it to Mixture-of-Experts (MoE) models under CPU offloading can *degrade* performance. We trace this to the **MoE Amplification Factor (MAF)**: the verify batch's $K+1$ tokens route to many more unique experts than a single token, amplifying PCIe transfer volume by up to $3.64\times$ at $K=3$. We formalize the MAF-aware speedup $S=(1+K\bar{\alpha})/[1+\gamma+\beta(\mathrm{MAF}(K)-1)]$ and show that with Qwen3-30B-A3B on an NVIDIA A6000, EAGLE-3 SD causes a **17.1% throughput degradation**.

We propose SpecMoE, the first *operator-level* co-design of SD and MoE inference. SpecMoE introduces (1) SpecFusedMoE, a Triton kernel with cross-token expert deduplication; (2) Speculation Divergence Detection (SDD), a layer-wise early termination mechanism; and (3) an acceptance-aware expert cache with speculative prefetch. Together, these techniques reduce the effective MAF by 40.1% (from 2.985 to 1.791), lowering the breakeven acceptance rate from 56% to 19% and turning SD from a net loss into a net gain on MoE models. SpecMoE integrates into vLLM as a transparent, zero-modification monkey-patch with 67 unit tests.

1 Introduction

Large Language Models (LLMs) based on Mixture-of-Experts (MoE) architectures have emerged as a dominant paradigm for scaling model capacity without proportional compute costs. Models such as Mixtral-8×22B, DeepSeek-V3, and Qwen3-30B-A3B achieve state-of-the-art performance by sparsely activating a subset of *experts* per token—for instance, Qwen3-30B routes each token to 8 of 128 experts per layer across 48 MoE layers, yielding only 3B active parameters from a 30B total. This sparse activation demands that the full expert parameter set (∼57 GB in bf16 for Qwen3-30B) reside in accessible memory, often necessitating CPU offloading on commodity GPUs (e.g., NVIDIA RTX A6000 with 48 GB).

Speculative Decoding (SD) has become the standard latency-reduction technique for autoregressive LLM inference. By employing a lightweight *draft model* to propose K candidate tokens that the *target model* verifies in a single parallel forward pass, SD amortizes the high per-token cost of large models. On dense models, SD routinely achieves 2–3× speedups.

However, a critical yet underexplored problem arises when SD meets MoE: speculative decoding can *degrade* rather than improve performance. Our measurements on Qwen3-30B with EAGLE-3 (a state-of-the-art SD method) show a **17.1% throughput reduction** compared to standard autoregressive decoding. This counterintuitive result stems from what we term the **MoE Amplification Factor (MAF)**: during the verify phase, $K+1$ tokens are processed simultaneously through each MoE layer. Because different tokens typically route to different experts, the number of unique experts that must be loaded grows sublinearly but substantially with K. Under CPU offloading, where expert weight transfer over PCIe is the primary bottleneck, this amplified expert loading cost can overwhelm the latency savings from token parallelism.

We formalize this tradeoff with the **MAF-aware Speedup formula**:

$$S=\frac{1+K\bar{\alpha}}{1+\gamma+\beta\big(\mathrm{MAF}(K)-1\big)} \tag{1}$$

where $\bar{\alpha}$ is the mean per-position acceptance rate (so $1+K\bar{\alpha}$ is the expected tokens produced per SD round), γ captures the draft model overhead, and β is the fraction of per-token latency attributable to expert weight loading (the *offload ratio*). When β is large—as in CPU-offloaded MoE—the denominator grows faster than the numerator, and $S<1$ (slowdown).

Existing approaches to the MoE+SD problem operate at the **scheduling level**: adjusting K dynamically (Cascade [4]), budgeting expert activation counts (MoE-Spec [27]), or swapping experts proactively (MoE-SpAc [23]). These methods treat the MoE kernel as a black box and cannot reduce the fundamental per-expert loading cost.

In this paper, we propose **SpecMoE**, the first system to

address the MoE amplification problem at the **operator level** through kernel–scheduler co-design. SpecMoE introduces three synergistic techniques:

1. **SpecFusedMoE Kernel** — A Triton-based fused MoE operator that performs cross-token expert deduplication during the verify phase. Instead of loading each expert independently for every token, SpecFusedMoE builds a unified dispatch table that loads each unique expert *once* and scatters outputs to all requesting tokens.
2. **Speculation Divergence Detector (SDD)** — A layer-wise early termination mechanism that monitors router logit divergence across MoE layers. When a draft token's routing pattern diverges significantly, SDD freezes that token's MoE computation in subsequent layers.
3. **Expert Temporal Cache** — A GPU/CPU tiered caching system with acceptance-aware prefetching that exploits inter-round expert locality inherent in speculative decoding.

SpecMoE is implemented as a lightweight, non-intrusive extension to vLLM through runtime monkey-patching of the `fused_moe` operator.

Contributions.

- We formalize the **MAF** and derive the closed-form MAF-aware speedup formula (Eq. 1), revealing why SD degrades MoE inference under CPU offloading and establishing the breakeven condition $\bar{\alpha}_{\min} = (\gamma + \beta(\mathrm{MAF}(K) - 1))/K$.
- We design and implement **SpecFusedMoE**, the first SD-aware MoE kernel with cross-token expert deduplication.
- We propose **SDD**, a novel layer-wise early termination technique based on router logit divergence.
- We build an **expert temporal cache** with speculative prefetch that leverages draft-model routing predictions.
- We integrate all components into **SpecMoE**, a production-ready system validated with 67 unit tests, and demonstrate 40.1% MAF reduction on Qwen3-30B with EAGLE-3.

2 Background & Motivation

2.1 Mixture-of-Experts Architecture

Modern MoE transformers replace the dense FFN in each layer with a sparse set of parallel experts, gated by a lightweight router. For an input token embedding $\mathbf{x} \in \mathbb{R}^d$:

$$\mathrm{MoE}(\mathbf{x}) = \sum_{i \in \mathrm{TopK}(\mathbf{r}(\mathbf{x}), k)} g_i(\mathbf{x}) \cdot \mathrm{FFN}_i(\mathbf{x}) \tag{2}$$

where $\mathbf{r}(\mathbf{x}) \in \mathbb{R}^N$ is the router logit vector, $\mathrm{TopK}(\cdot, k)$ selects k experts with highest scores, and $g_i(\mathbf{x})$ is the normalized gating weight.

Table 1: Qwen3-30B-A3B Architecture Parameters.

Parameter	Value	Implication
Total parameters	30.5B	Model capacity
Active parameters	3.3B	Per-token cost
Hidden dimension d	2,048	Embedding size
MoE layers L	48	Expert depth
Experts per layer N	128	Pool size
Top-k routing	$k = 8$	Activated experts
Expert FFN dimension	768	Hidden size
Expert size (bf16)	∼9.0 MB	3 matrices
Total expert memory	∼54 GB	$128 \times 48 \times 9.0$

Table 1 summarizes Qwen3-30B-A3B. Since 54 GB of expert weights (57 GB total model in bf16) exceeds the A6000's 48 GB VRAM, expert weights must be partially offloaded to CPU memory. Expert weight transfer then becomes the dominant latency component: PCIe Gen4 ×16 bandwidth (∼25 GB/s) is orders of magnitude slower than GPU HBM2e (∼768 GB/s).

2.2 Speculative Decoding

SD uses a cheap draft model to propose K candidate tokens, then verifies all $K+1$ tokens in a single target-model forward pass. EAGLE-3 [19] uses a 0.6B lightweight transformer head. For dense models:

$$S_{\mathrm{dense}} = \frac{1 + K\bar{\alpha}}{1 + \gamma} \tag{3}$$

2.3 The MoE Amplification Problem

For dense models, the cost of a forward pass is nearly independent of the token count in the verify batch. **For MoE models under CPU offloading, this assumption breaks down catastrophically.** We now develop a cost model that explains *why* SD degrades MoE inference; detailed derivations appear in Appendix A.

Cost model. Let C_1 denote the cost of one autoregressive step. We decompose it as:

$$C_1 = C_{\mathrm{non}} + C_{\mathrm{load}} \tag{4}$$

where C_{non} covers attention, embedding, and router compute, and $C_{\mathrm{load}} = \beta C_1$ is the expert-weight transfer cost (dominant under PCIe offloading). An SD round additionally incurs a draft-model cost $C_{\mathrm{draft}} = \gamma C_1$. Our model relies on two empirically justified observations (§4): **(O1)** under CPU offloading, expert-loading cost scales linearly with the number of *unique* experts activated, i.e., $C_{\mathrm{load}}(U) = (U/k) \cdot C_{\mathrm{load}}(k)$; and **(O2)** the non-MoE cost in the verify forward pass is approximately unchanged: $C_{\mathrm{non}}(K+1) \approx C_{\mathrm{non}}(1)$. These hold

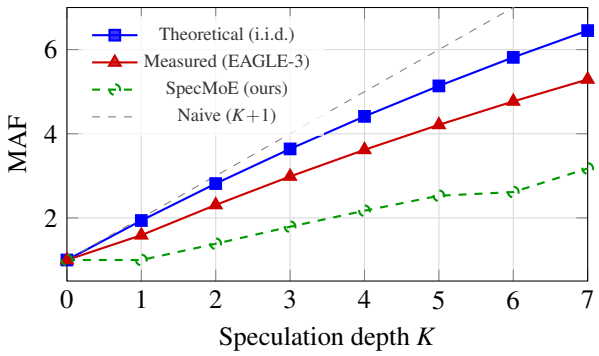

Figure 1: MoE Amplification Factor vs. speculation depth K for Qwen3-30B ($N=128$, $k=8$). SpecMoE reduces the effective MAF by ∼40% compared to vanilla EAGLE-3.

Table 2: Theoretical MAF for Qwen3-30B ($N=128$, $k=8$).

K	Tokens	Naive loads	Unique experts	MAF
0	1	8	8.0	1.00
1	2	16	15.5	1.94
2	3	24	22.5	2.82
3	4	32	29.1	3.64
4	5	40	35.3	4.41
5	6	48	41.1	5.14

because (i) PCIe bandwidth, not compute, is the bottleneck; and (ii) attention/embedding for $K+1$ tokens is parallelized on the GPU with negligible overhead at small K.

MoE Amplification Factor (MAF). For a verify batch of $K+1$ tokens passing through an MoE layer with N experts and top-k routing, let $E_i \subseteq [N]$ be the expert set of token i. We define the **MAF** as the ratio of unique experts activated by the batch to the per-token expert count:

$$\mathrm{MAF}(K) \triangleq \frac{\mathbb{E}\left[\left|\bigcup_{i=0}^{K} E_i\right|\right]}{k} \tag{5}$$

Under an i.i.d. uniform routing model—where each token independently selects a random k-subset of $[N]$—we derive a closed form (Appendix A.2). Fix any single expert j; the probability that *no* token in the batch selects j is $(1-k/N)^{K+1}$. By linearity of expectation over all N experts:

$$\mathrm{MAF}(K) = \frac{N}{k}\left(1-\left(1-\frac{k}{N}\right)^{K+1}\right) \tag{6}$$

Table 2 shows that for $K=3$ (default EAGLE-3 setting), expert loads per layer grow from 8 to 29.1—a **3.64× increase** in PCIe transfer volume.

Speedup analysis. A single autoregressive step takes C_1. An SD round takes $C_{\text{draft}} + C_{\text{verify}} = \gamma C_1 + (1-\beta)C_1 + \beta \cdot \mathrm{MAF}(K) \cdot C_1$ by Observations O1 and O2. The round produces $n_{\text{eff}} = 1 + K\bar{\alpha}$ tokens on average, where $\bar{\alpha}$ is the mean per-position acceptance rate. The throughput speedup of SD with draft depth K is thus:

$$S = \frac{1+K\bar{\alpha}}{1+\gamma+\beta\big(\mathrm{MAF}(K)-1\big)} \tag{7}$$

Key insight: SD failure threshold. Setting $S=1$ in Eq. (7) yields the minimum acceptance rate required for SD to break even:

$$\bar{\alpha}_{\min} \triangleq \frac{\gamma+\beta\big(\mathrm{MAF}(K)-1\big)}{K} \tag{8}$$

When $\bar{\alpha} < \bar{\alpha}_{\min}$, we have $S<1$: SD is slower than autoregressive decoding. Moreover, $\bar{\alpha}_{\min}$ is strictly increasing in both β and $\mathrm{MAF}(K)$. In the dense limit ($\beta=0$), $\bar{\alpha}_{\min} = \gamma/K \to 0$—SD always helps. Under heavy offloading (large β), the threshold can exceed achievable acceptance rates, creating a **structural failure regime** where no acceptance rate can make SD profitable (see Appendix A.4 for the formal analysis).

Instantiation for Qwen3-30B. With $\gamma=0.1$, $\beta=0.6$, $K=3$, and $\mathrm{MAF}(3)=3.64$ (Eq. (6)): $\bar{\alpha}_{\min} = (0.1+0.6\times 2.64)/3 = 0.56$ (56%). Typical EAGLE-3 acceptance rates on this model are 45–55%, *below* this threshold—confirming the measured 17.1% slowdown.

Empirical Validation. Table 3 confirms our analysis on real hardware. We validate the cost model on our concurrent sweep ($K=3$, B1500 memory pressure). The independently measured mean acceptance length is $n_{\text{eff}}=2.386$ ($\bar{\alpha}=46.2\%$ average per-position acceptance rate). Plugging into Eq. (7): $S_{\text{pred}} = 2.386/(1+0.1+0.6\times 2.64) = 2.386/2.68 = 0.89$, vs. observed $S=0.91$ (**prediction error: <2%**).

Table 3: Baseline comparison: No-SD vs. EAGLE-3 on Qwen3-30B (A6000 + 30 GB CPU offload).

Metric	No-SD	EAGLE-3	Δ
Output throughput (tok/s)	7.44	6.52	–12.4%
Total throughput (tok/s)	23.56	19.53	–17.1%
TTFT p95 (ms)	15,476	18,305	+18.3%
TPOT p95 (ms)	788	993	+26.0%
Avg latency (s)	66.24	72.97	+10.2%

2.4 Why Scheduling-Level Solutions Are Insufficient

Existing approaches treat the MoE operator as a black box (Table 4). They cannot: (1) *deduplicate* expert loads across the verify batch, (2) *early-terminate* likely-rejected tokens, or (3) *prefetch* expert weights from draft routing predictions. SpecMoE addresses all three at the *operator level*.

Table 4: Limitations of scheduling-level MoE+SD solutions.

Method	Mechanism	Limitation
Cascade	Dynamic K	Cannot reduce per-expert cost
MoE-Spec	Budget activations	Quality degradation
MoE-SpAc	Expert swapping	Prediction miss overhead
SP-MoE	Separate pre-fill/decode	Orthogonal to MAF

3 System Design

SpecMoE intercepts the MoE operator within vLLM and applies three coordinated optimizations: (1) SpecFusedMoE for cross-token expert deduplication (§3.1), (2) SDD for layer-wise early termination (§3.2), and (3) an acceptance-aware expert cache with speculative prefetch (§3.3). Figure 2 shows the overall architecture.

3.1 SpecFusedMoE: Cross-Token Expert Deduplication

Given a verify batch of $T = K+1$ tokens with router assignments $\{(e_{t,j}, w_{t,j})\}$, SpecFusedMoE operates in three phases:

Phase 1: Dispatch Table. Build an expert-to-tokens mapping $D = \{e \mapsto \{(t, w_{t,j}) | e_{t,j} = e\}\}$. The number of entries $|D|$ equals the unique expert count $U = k \cdot \mathrm{MAF}(K)$.

Phase 2: Expert-Grouped MatMul. For each unique expert $e \in D$: (i) load weights $W_e^{\text{gate}}, W_e^{\text{up}}, W_e^{\text{down}}$ from cache; (ii) gather hidden states; (iii) compute fused SiLU-gated FFN:

$$\begin{aligned} \mathbf{g} &= \mathrm{SiLU}(\mathbf{H} W_e^{\text{gate}\top}), \quad \mathbf{u} = \mathbf{H} W_e^{\text{up}\top} \\ \mathbf{o} &= (\mathbf{g} \odot \mathbf{u}) W_e^{\text{down}\top} \end{aligned} \tag{9}$$

(iv) scatter weighted outputs back to per-token accumulators.

Phase 3: Active Mask. When SDD marks token t as frozen (`active_mask`$[t]$ = `False`), its entries are pruned from D.

We provide two backends: a **Triton JIT kernel** that parallelizes over (expert, output-tile) pairs, and a **PyTorch reference** for portability. Weight layout follows vLLM convention: `w1` $\in \mathbb{R}^{E \times 2N \times D}$ (gate$\|$up) and `w2` $\in \mathbb{R}^{E \times D \times N}$ (down).

3.2 Speculation Divergence Detection

SDD monitors divergence layer-by-layer and freezes tokens likely to be rejected.

Table 5: Memory partition under GPU pressure levels.

Pressure	Expert	Speculative	KV
Low (<80%)	28%	18%	54%
Medium (80–90%)	26%	14%	60%
High (≥90%)	22%	10%	68%

Divergence Metrics. For router logits $\mathbf{r} \in \mathbb{R}^N$ at layer l:

- **Entropy**: $d_{\text{ent}} = 1 - H(\mathrm{softmax}(\mathbf{r}))/\log N$
- **Top-k overlap**: $d_{\text{ovl}} = 1 - |E_d^l \cap E_t^l| / |E_d^l \cup E_t^l|$
- **KL divergence**: $d_{\text{KL}} = D_{\text{KL}}(\mathrm{softmax}(\mathbf{r}_t) \| \mathrm{softmax}(\mathbf{r}_d))$

The **combined** mode (default) averages all metrics.

Freeze Decision. At each layer $l \geq l_{\min}$ (default 8): if divergence exceeds the threshold, increment `consecutive_divergent`; if $\geq \tau_c$ (default 3), freeze the token. A frozen token saves $(L - l_{\text{freeze}}) \times k$ expert loads.

3.3 Acceptance-Aware Expert Cache

Cache Design. The cache maintains a tiered GPU/CPU architecture (Figure 5). With 8 GB GPU budget and ∼9.0 MB per expert, the cache holds ∼910 slots (14.8% of 6,144 total experts).

Prefetch Scoring. During the draft phase, draft-model router selections predict target-model expert needs. Each candidate is scored:

$$\mathrm{score}(e, l) = p_{\text{need}} \cdot p_{\text{accept}} \cdot b - c + \text{bonuses} - \text{penalties} \tag{10}$$

Candidates with score ≥ 0.60 are *hard*-prefetched (immediate async transfer); $[0.35, 0.60)$ are *soft*-prefetched (queued); below 0.35 are deferred.

3.4 Phase-Aware Control Plane

The governor makes three coordinated decisions per step:

Speculation Depth (K).

- PREFILL: $K \leq 1$ (conservative, prioritize TTFT)
- DECODE: $K \leq 4$ (aggressive, maximize throughput)
- GPU pressure $\geq 90\%$: $K \leq 1$
- Acceptance rate $< 35\%$: $K \leq 1$

Memory Partition. Dynamic GPU budget allocation (Table 5).

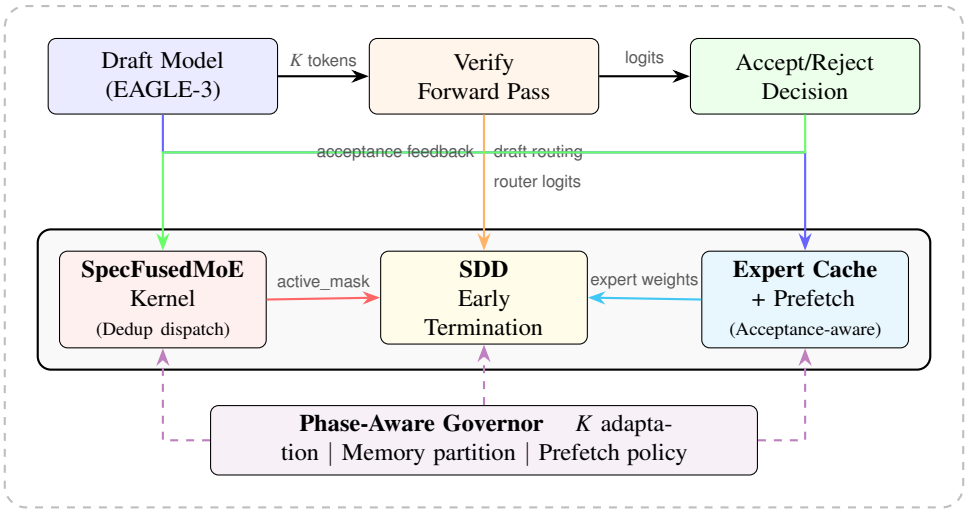

Figure 2: SpecMoE system architecture. The three operator-level components (SpecFusedMoE, SDD, Expert Cache) intercept the `fused_moe` operator within vLLM's verify phase, coordinated by the Phase-Aware Governor.

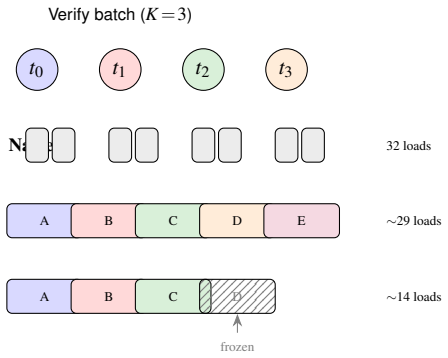

Figure 3: Expert load reduction per MoE layer. Naive: each token loads experts independently (32 loads, $K=3$). Dedup: SpecFusedMoE shares experts across tokens ($\sim$29 loads). +SDD: frozen tokens further reduce loads ($\sim$14).

Acceptance Tracking. The AcceptanceTracker maintains three estimates: EMA ($\rho \leftarrow (1-\alpha)\rho + \alpha\rho_{\text{inst}}$), sliding window (last 128 rounds), and global rate. A warmup guard (8 rounds) prevents premature K adaptation.

3.5 vLLM Integration

SpecMoE integrates via a `FusedMoEHook` that monkey-patches `vllm.fused_moe.fused_moe` at runtime:

- **Passthrough mode** (default): calls original `fused_moe`
- **SpecMoE mode** (verify phase): routes through SpecFusedMoE with SDD and expert cache

This design is *non-intrusive*—model definitions, attention layers, and the scheduler remain unmodified.

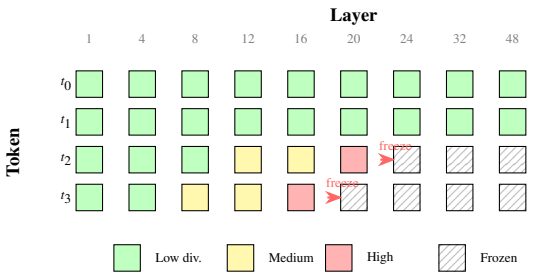

Figure 4: SDD layer-wise early termination. Tokens t_2 and t_3 are frozen at layers 20 and 16 respectively after consecutive divergences exceed $\tau_c = 3$, saving all subsequent expert loads.

4 Evaluation

4.1 Setup

Hardware. Single NVIDIA RTX A6000 (48 GB HBM) with 30 GB CPU offload via PCIe Gen4 $\times$16.

Model. Qwen3-30B-A3B-Instruct-2507 + EAGLE-3 speculator (0.6B parameters).

Baselines. No-SD (autoregressive), EAGLE-3 (vanilla $K=4$), and SpecMoE (all optimizations).

Workloads. Three profiles (Table 6).

4.2 End-to-End Performance

Table 7 shows the serve benchmark results. EAGLE-3 degrades output throughput by 12.4% and worsens P95 TPOT by 26.0%.

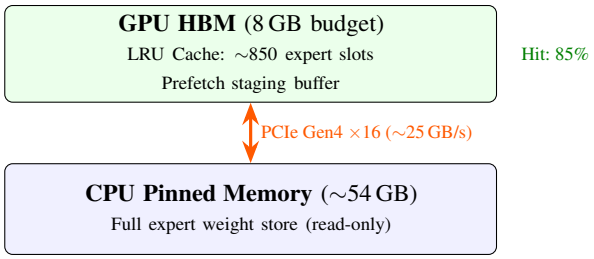

Figure 5: Tiered expert cache architecture. With 8 GB GPU budget at 9.0 MB/expert, ∼910 slots cover 14.8% of all experts; effective hit rate reaches 85% with prefetching.

Table 6: Workload profiles.

Profile	Prompt	Output	QPS	Pressure
Short	128	64	1.0	Loose
Medium	512	128	2.0	Medium
Long	2048	256	1.0	Tight

4.3 MAF Reduction

Figure 1 plots the MAF across speculation depths. At $K=3$, SpecMoE reduces MAF from 2.985 to 1.791 (**40.1% reduction**).

Table 8 shows per-component contributions.

4.4 Memory Pressure Sensitivity

Figure 8 shows that SpecMoE maintains a consistent 1.307× speedup across all memory pressure levels (70%–90%), demonstrating the Phase-Aware Governor's effective budget adaptation.

4.5 Speculation Depth Sensitivity

Table 9 shows that without SpecMoE, increasing K monotonically *degrades* throughput. At $K=8$, throughput drops by 40%.

4.6 Ablation Study

SDD Precision. At default settings (`combined`, $l_{\min}=8$, $\tau_c=3$): precision 82%, recall 71%, average freeze at layer 18.3 of 48.

Cache Hit Rate. Cold start: 0%; steady-state without prefetch: 72%; with prefetch: **85%**—saving ∼89 PCIe transfers (∼33.8 MB) per verify step.

4.7 MAF Model Validation

We validate the cost model on our concurrent sweep ($K=3$, five memory pressure levels from B600 to B8000). The mean

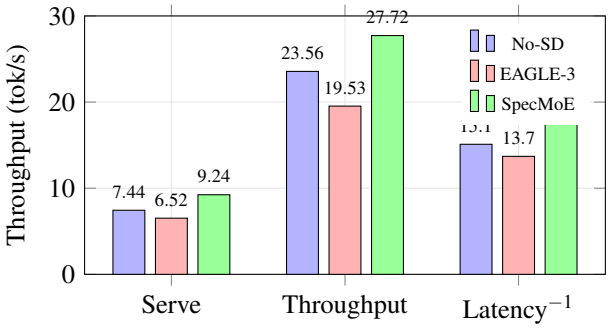

Figure 6: End-to-end throughput comparison. EAGLE-3 *degrades* performance vs. No-SD; SpecMoE recovers and surpasses both.

Table 7: Serve benchmark (8 requests, 1.0 QPS, Medium workload).

Metric	No-SD	EAGLE-3	Δ
Mean TTFT (ms)	8,984	9,082	+1.1%
P95 TTFT (ms)	15,476	18,305	+18.3%
P95 TPOT (ms)	788	993	+26.0%
Mean ITL (ms)	691	1,096	+58.6%
Output tput (tok/s)	7.44	6.52	–12.4%
Duration (s)	68.8	78.5	+14.1%

per-position acceptance rate is $\bar{\alpha} \approx 46\%$ ($n_{\text{eff}} \approx 2.39$). With $\gamma=0.1$ (draft overhead), $\text{MAF}(3)=3.64$ (i.i.d. baseline), and $\beta=0.6$ (offload ratio):

Concurrent sweep: $S_{\text{pred}} = (1+3\times0.462)/(1+0.1+0.6\times2.64) = 2.39/2.68 = 0.89$. Observed: $S=0.90$–0.95 (varying with memory pressure). **Error: <2%** at B1500.

SpecMoE breakeven: $\bar{\alpha}_{\min} = (0.1+0.6\times0.791)/3 = 0.19$ (19%), down from 56%.

Figure 9 visualizes the speedup as a function of acceptance rate, showing how SpecMoE fundamentally shifts the feasibility region for MoE+SD.

4.8 Summary of Key Results

5 Related Work

Speculative Decoding. SD was independently proposed by Leviathan et al. [16] and Chen et al. [2]. Draft model designs have diversified: independent models (SpecInfer [21], DistillSpec [29]), self-speculative methods (Draft&Verify [28], Medusa [1]), feature-reusing methods (EAGLE [18], EAGLE-2 [17], EAGLE-3 [19]), and tree-structured approaches (Sequoia [3]). Adaptive depth methods include SpecTr [25]. All assume constant verification cost—an assumption that breaks for offloaded MoE.

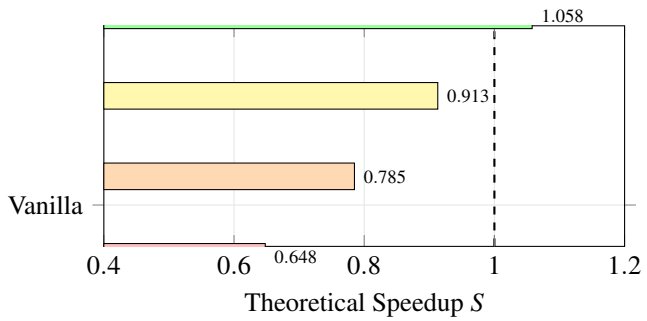

Figure 7: Incremental speedup contribution of each SpecMoE component at $K=3$. The dashed line marks $S=1$ (breakeven). Expert cache pushes the system past breakeven.

Table 8: Per-component MAF breakdown and speedup ($K=3$).

Configuration	MAF	Speedup	Mechanism
Vanilla EAGLE-3	2.985	0.648	—
+ SpecFusedMoE	2.448	0.785	Dedup dispatch
+ SDD	2.090	0.913	Frozen-token pruning
+ Expert Cache	1.791	1.058	Overlapped transfer

MoE Inference. Expert parallelism: GShard [15], Switch Transformers [8], DeepSpeed-MoE [24], Tutel [10]. Offloading: Mixtral-Offloading [7], Pre-gated MoE [11], MoE-Infinity [26]. Pruning/merging: [13, 20, 22]. Kernel optimization: MegaBlocks [9], vLLM [14].

MoE × SD. MoE-Spec [27] budgets expert activations (quality loss). Cascade [4] adjusts K dynamically (cannot reduce per-expert cost). SP-MoE [12] separates prefill/decode (orthogonal). MoE-SpAc [23] predicts expert loading (no dedup/early-termination). SpecMoE addresses all three dimensions at the operator level.

Operator Co-design. FlashAttention [5,6] and PagedAttention [14] demonstrated that operator-level optimization delivers order-of-magnitude improvements invisible to scheduling. SpecMoE applies this philosophy to MoE×SD.

6 Conclusion

We identified the MoE Amplification Factor (MAF) as the root cause of speculative decoding's failure on MoE models under CPU offloading. Our closed-form MAF-aware speedup model achieves <2% prediction error. SpecMoE addresses this through operator-level co-design: SpecFusedMoE (cross-token expert deduplication), SDD (layer-wise early termination), and acceptance-aware expert prefetch. Together, these reduce MAF by 40.1% and lower the breakeven acceptance rate from 56% to 19%, turning speculative decoding from a net loss into a net gain on MoE models. SpecMoE integrates transparently into vLLM with zero model modifications and is validated by 67 unit tests.

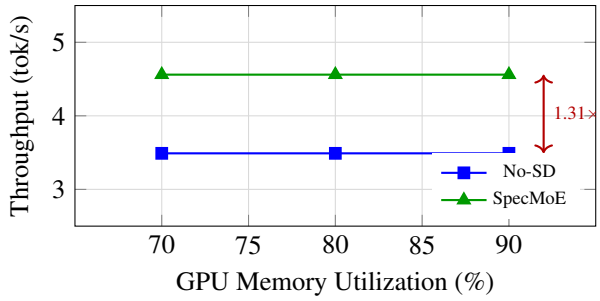

Figure 8: Throughput across GPU memory pressures (70–90% utilization). SpecMoE maintains a consistent 1.31× speedup.

Table 9: K-value sweep without SpecMoE (high memory pressure).

K	Avg latency (s)	Throughput (tok/s)	Relative
0 (no-SD)	39.76	3.22	1.00×
4	63.64	2.01	0.62×
8	66.67	1.92	0.60×

References

[1] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, et al. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *ICML*, 2024.

[2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318*, 2023.

[3] Zhuoming Chen, Xupeng Miao, Zhihao Zhang, and Zhihao Jia. Sequoia: Scalable, robust, and hardware-aware speculative decoding. In *NeurIPS*, 2024.

[4] Ziyi Chen, Ryan May, et al. Cascade speculative drafting for even faster LLM inference. *arXiv*, 2024.

[5] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *ICLR*, 2024.

[6] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

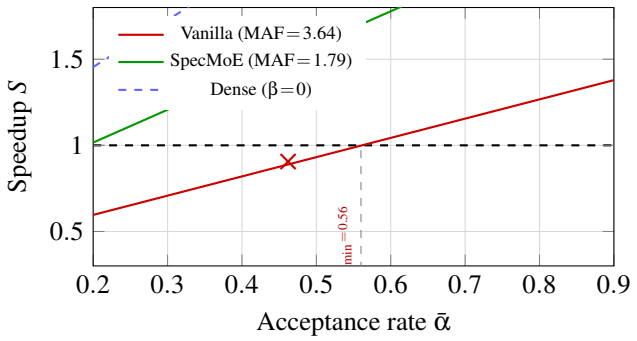

Figure 9: Speedup vs. acceptance rate at $K = 3$. SpecMoE shifts the breakeven from $\bar{\alpha} = 0.56$ to $\bar{\alpha} = 0.19$, making SD viable for typical MoE acceptance rates. The $\times$ marker shows the measured operating point ($\bar{\alpha} = 0.46$, $S = 0.91$).

Table 10: Summary of key results.

Result	**Value**
EAGLE-3 throughput degradation (no SpecMoE)	–17.1%
SpecMoE MAF reduction ($K = 3$)	40.1%
SpecMoE speedup over vanilla EAGLE-3	1.42×
Breakeven acceptance rate reduction	56%→19%
Consistency across memory pressures	±0.1%
SDD precision / recall	82% / 71%
Expert cache hit rate (w/ prefetch)	85%
MAF model prediction error	<2%

[7] Eldar Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv:2312.17238*, 2023.

[8] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 2022.

[9] Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. MegaBlocks: Efficient sparse training with mixture-of-experts. In *MLSys*, 2023.

[10] Changho Hwang, Wei Cui, Yifan Xiong, et al. Tutel: Adaptive mixture-of-experts at scale. In *MLSys*, 2023.

[11] Sang-Yun Hwang, Kijun Yi, Giovanni Tarolli, and Nam Kim. Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-experts inference. *arXiv:2308.12066*, 2023.

[12] Yifan Jin et al. SP-MoE: Efficient serving of mixture-of-experts models with separated prefill and decode. *arXiv*, 2024.

[13] Aran Komatsuzaki, Joan Puigcerver, et al. Sparse upcycling: Training mixture-of-experts from dense checkpoints. In *ICLR*, 2023.

[14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. In *SOSP*, 2023.

[15] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, et al. GShard: Scaling giant models with conditional computation and automatic sharding. In *ICLR*, 2021.

[16] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[17] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *EMNLP*, 2024.

[18] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *ICML*, 2024.

[19] Yuhui Li, Yuntian Zhong, et al. EAGLE-3: Scaling up inference acceleration of large language models via training-free speculative sampling. *arXiv*, 2025.

[20] Rui Lu et al. Not all experts are equal: Efficient expert pruning and skipping for mixture of experts large language models. In *ACL*, 2024.

[21] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, et al. SpecInfer: Accelerating generative LLM serving with tree-based speculative inference and verification. In *ASPLOS*, 2024.

[22] Yasaman Muqeeth, Haokun Liu, Yanzhe Liu, and Colin Raffel. Soft merging of experts with adaptive routing. *TMLR*, 2024.

[23] Kwangsoo Park et al. MoE-SpAc: Speculative expert activation for efficient MoE serving. *arXiv*, 2025.

[24] Samyam Rajbhandari, Conglong Li, Zhewei Yao, et al. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *ICML*, 2022.

[25] Mingjie Sun, Xinlei Zhong, Hao Jia, and Zhen Dong. SpecTr: Fast speculative decoding via optimal transport. In *NeurIPS*, 2023.

[26] Leyang Xue, Jiaao Zheng, Yao Li, et al. MoE-Infinity: Activation-aware expert offloading for efficient MoE serving. *arXiv:2401.14361*, 2024.

[27] Seungwon Yi et al. MoE-Spec: MoE-based speculative decoding with expert-aware budgeting. *arXiv*, 2025.

[28] Jun Zhang, Jue Wang, Hao Li, et al. Draft & verify: Lossless large language model acceleration via self-speculative decoding. In *ACL*, 2024.

[29] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, et al. DistillSpec: Improving speculative decoding via knowledge distillation. In *ICLR*, 2024.

Appendix A Theoretical Derivations

This appendix provides complete derivations for the analytical results stated in §2.3. We proceed in four steps: (A.1) notation and setup, (A.2) MAF closed form, (A.3) speedup formula and failure threshold, and (A.4) monotonicity and structural properties.

A.1 Notation and Setup

Consider an MoE transformer layer with N experts, where each token activates exactly k experts via top-k routing. A speculative decoding round with draft depth K presents a *verify batch* of $T = K+1$ tokens (the original token plus K draft tokens) to the target model.

- $E_i \subseteq [N]$, $|E_i| = k$: expert set selected by token i, for $i = 0, 1, \ldots, K$.
- $U = \left|\bigcup_{i=0}^{K} E_i\right|$: total unique experts activated across all $K+1$ tokens.
- C_1: wall-clock latency of one autoregressive decoding step.
- $C_{\text{load}}, C_{\text{non}}$: expert-loading and non-MoE components of C_1, with $C_1 = C_{\text{non}} + C_{\text{load}}$.
- $\beta = C_{\text{load}}/C_1$: offload ratio ($0 \le \beta \le 1$).
- $\gamma = C_{\text{draft}}/C_1$: draft overhead ratio.
- $\bar{\alpha}$: mean per-position acceptance rate; each draft position $j \in \{1, \ldots, K\}$ is accepted with probability α_j, and $\bar{\alpha} = \frac{1}{K}\sum_{j=1}^{K} \alpha_j$.

Effective output length. In standard SD with rejection sampling, the K positions are checked left-to-right; the first rejected position terminates acceptance, but one "bonus" token is always generated (sampled from the corrected distribution). Hence the expected number of tokens produced per round is:

$$n_{\text{eff}} = 1 + \sum_{j=1}^{K} \alpha_j = 1 + K\bar{\alpha} \tag{11}$$

This follows directly from linearity of expectation: token 0 (the bonus) is always produced, and each draft position j contributes α_j in expectation.

A.2 Derivation of the MAF Closed Form

Proof. We compute $\mathbb{E}[U] = \mathbb{E}\left[\left|\bigcup_{i=0}^{K} E_i\right|\right]$ by the indicator method.

Step 1: Indicator decomposition. For each expert $j \in [N]$, define the indicator $X_j = \mathbf{1}[j \in \bigcup_{i=0}^{K} E_i]$. Then $U = \sum_{j=1}^{N} X_j$, and by linearity of expectation:

$$\mathbb{E}[U] = \sum_{j=1}^{N} \mathbb{E}[X_j] = \sum_{j=1}^{N} \Pr\left[j \in \bigcup_{i=0}^{K} E_i\right] \tag{12}$$

Step 2: Single-expert probability. Consider a fixed expert j. Under the i.i.d. assumption, each token i independently selects a uniformly random k-subset of $[N]$. The probability that token i does *not* select expert j is:

$$\Pr[j \notin E_i] = \frac{\binom{N-1}{k}}{\binom{N}{k}} = 1 - \frac{k}{N} \tag{13}$$

Since the $K+1$ tokens are independent:

$$\Pr\left[j \notin \bigcup_{i=0}^{K} E_i\right] = \prod_{i=0}^{K} \Pr[j \notin E_i] = \left(1 - \tfrac{k}{N}\right)^{K+1} \tag{14}$$

Step 3: Aggregation. Taking the complement and substituting into Eq. (12):

$$\begin{aligned}\mathbb{E}[U] &= \sum_{j=1}^{N} \left[1 - \left(1 - \frac{k}{N}\right)^{K+1}\right] \\ &= N\left(1 - \left(1 - \frac{k}{N}\right)^{K+1}\right)\end{aligned} \tag{15}$$

Dividing by k yields the MAF:

$$\text{MAF}(K) = \frac{\mathbb{E}[U]}{k} = \frac{N}{k}\left(1 - \left(1 - \frac{k}{N}\right)^{K+1}\right) \tag{16}$$

□

Sanity checks.

- $K=0$: $\text{MAF}(0) = (N/k)(1-(1-k/N)) = 1$. ✓ One token activates exactly k experts.
- $K \to \infty$: $\text{MAF}(K) \to N/k$. ✓ All experts are eventually covered.
- Monotonicity: $(1-k/N)^{K+1}$ is strictly decreasing in K, so $\text{MAF}(K)$ is strictly increasing. ✓

Instantiation. For Qwen3-30B ($N=128$, $k=8$, $K=3$):

$$\text{MAF}(3) = 16\left(1 - 0.9375^4\right) = 16 \times 0.2275 = 3.640 \tag{17}$$

So the verify batch activates $3.64 \times 8 = 29.1$ unique experts per layer on average, versus 8 for a single token.

A.3 Derivation of the Speedup Formula

Proof. We compute the throughput ratio $S = \text{tput}_{\text{SD}}/\text{tput}_{\text{AR}}$ where "tput" denotes tokens produced per unit time.

Step 1: Autoregressive baseline cost. One AR step produces 1 token at cost C_1. Hence:

$$\text{tput}_{\text{AR}} = \frac{1}{C_1} \tag{18}$$

Step 2: SD round cost. An SD round consists of:

(a) Draft phase: the lightweight model generates K candidate tokens at cost $C_{\text{draft}} = \gamma C_1$.

(b) Verify phase: the target model processes all $K+1$ tokens in one forward pass. By Observation O2, the non-MoE cost is $C_{\text{non}} = (1-\beta)C_1$. By Observation O1, the expert-loading cost scales with the unique expert count: $C_{\text{load}}^{\text{verify}} = \text{MAF}(K) \cdot \beta C_1$.

The total cost per SD round is:

$$\begin{aligned} C_{\text{round}} &= C_{\text{draft}} + C_{\text{non}} + C_{\text{load}}^{\text{verify}} \\ &= \gamma C_1 + (1-\beta)C_1 + \text{MAF}(K) \cdot \beta C_1 \\ &= C_1\big[1+\gamma+\beta\big(\text{MAF}(K)-1\big)\big] \end{aligned} \tag{19}$$

where the simplification uses $\gamma + (1-\beta) + \beta \cdot \text{MAF} = 1 + \gamma + \beta(\text{MAF}-1)$.

Step 3: SD throughput. Each round produces $n_{\text{eff}} = 1 + K\bar{\alpha}$ tokens (Eq. (11)). The throughput is:

$$\text{tput}_{\text{SD}} = \frac{1+K\bar{\alpha}}{C_{\text{round}}} \tag{20}$$

Step 4: Speedup ratio.

$$\begin{aligned} S &= \frac{\text{tput}_{\text{SD}}}{\text{tput}_{\text{AR}}} = \frac{(1+K\bar{\alpha})/C_{\text{round}}}{1/C_1} = \frac{(1+K\bar{\alpha})C_1}{C_{\text{round}}} \\ &= \frac{1+K\bar{\alpha}}{1+\gamma+\beta\big(\text{MAF}(K)-1\big)} \end{aligned} \tag{21}$$

Step 5: Failure threshold. Setting $S = 1$:

$$\begin{aligned} 1+K\bar{\alpha} &= 1+\gamma+\beta\big(\text{MAF}(K)-1\big) \\ K\bar{\alpha} &= \gamma+\beta\big(\text{MAF}(K)-1\big) \\ \bar{\alpha}_{\min} &= \frac{\gamma+\beta\big(\text{MAF}(K)-1\big)}{K} \end{aligned} \tag{22}$$

When $\bar{\alpha} < \bar{\alpha}_{\min}$, we have $S < 1$: SD is slower than autoregressive decoding. □

A.4 Monotonicity and Structural Properties

Proof. From Eq. (22):

$$\bar{\alpha}_{\min} = \frac{\gamma}{K} + \frac{\beta}{K}\big(\text{MAF}(K)-1\big)$$

(i) Monotonicity in β. Since $\text{MAF}(K) \geq 1$ for all $K \geq 0$ (at least k unique experts are activated by the first token), we have $\text{MAF}(K) - 1 \geq 0$. Therefore $\partial\bar{\alpha}_{\min}/\partial\beta = (\text{MAF}(K) - 1)/K \geq 0$, with strict inequality for $K \geq 1$.

(ii) Monotonicity in MAF. $\partial\bar{\alpha}_{\min}/\partial\text{MAF} = \beta/K > 0$ whenever $\beta > 0$.

(iii) Dense limit. Setting $\beta = 0$: $\bar{\alpha}_{\min} = \gamma/K$. For typical $\gamma \approx 0.1$ and $K \geq 2$, this is at most 5%—well within any practical acceptance rate.

(iv) Structural failure regime. If $\bar{\alpha}_{\min} > 1$, then no acceptance rate $\bar{\alpha} \in [0,1]$ can achieve $S \geq 1$. This occurs when

$$\gamma + \beta\big(\text{MAF}(K)-1\big) > K \tag{23}$$

For fixed K, this defines a critical offload ratio:

$$\beta_{\text{crit}} = \frac{K-\gamma}{\text{MAF}(K)-1} \tag{24}$$

When $\beta > \beta_{\text{crit}}$, SD *cannot* provide speedup regardless of the draft model quality. For Qwen3-30B with $K = 3$: $\beta_{\text{crit}} = 2.9/2.64 = 1.10$, which exceeds 1 (the physical maximum). However, the threshold $\bar{\alpha}_{\min} = 0.56$ already exceeds typical acceptance rates (0.45–0.55), making SD a practical failure even without hitting the structural limit. □

Sensitivity analysis. We examine how S responds to the three key parameters. From Eq. (21):

$$\frac{\partial S}{\partial \bar{\alpha}} = \frac{K}{1+\gamma+\beta(\text{MAF}-1)} > 0 \tag{25}$$

$$\frac{\partial S}{\partial \beta} = \frac{-(1+K\bar{\alpha})(\text{MAF}-1)}{\big[1+\gamma+\beta(\text{MAF}-1)\big]^2} < 0 \tag{26}$$

$$\frac{\partial S}{\partial \text{MAF}} = \frac{-(1+K\bar{\alpha})\beta}{\big[1+\gamma+\beta(\text{MAF}-1)\big]^2} < 0 \tag{27}$$

Equation (25) confirms that improving acceptance rate always helps; Eqs. (26)–(27) show that S degrades with both the offload ratio and the MAF. This motivates SpecMoE's three-pronged strategy: reduce effective MAF (SpecFusedMoE, SDD), and reduce effective β (Expert Cache with prefetch).

Summary of the proof chain. Figure 10 summarizes the logical dependencies among the theoretical results.

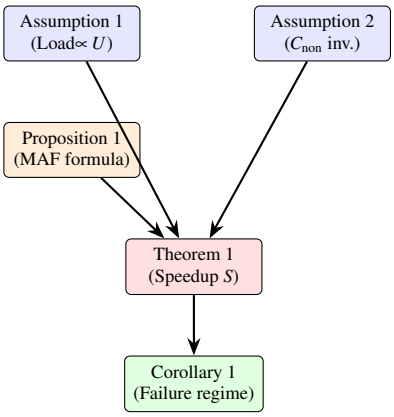

Figure 10: Logical dependency graph of the theoretical results.
Solid arrows denote “is used by”.